

DOC_CALIDAD.md — Calidad de Software

Proyecto ANA-VET · Sistema SaaS Veterinario

1. Introducción

Este documento analiza las prácticas de calidad de software implementadas en ANA-VET, incluyendo seguridad, manejo de errores, refactorización, buenas prácticas de código y métricas de calidad. Todo el análisis está basado en el código fuente real del proyecto.

2. Seguridad Implementada

2.1 Autenticación JWT (JSON Web Tokens)

Archivo: `src/middleware/authMiddleware.js`

El sistema implementa autenticación stateless mediante JWT con las siguientes características:

```
const JWT_SECRET = process.env.JWT_SECRET || 'saas_vet_secret_2026';
const SALT_ROUNDS = 10; // bcrypt rounds – balance seguridad/rendimiento

function authMiddleware(req, res, next) {
  const authHeader = req.headers['authorization'];

  // Validación del formato del header
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Token no proporcionado' });
  }

  const token = authHeader.split(' ')[1];
  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.user = decoded; // { id, tipo, clinica_id, nombre }
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Token inválido o expirado' });
  }
}
```

Características de seguridad:

- Tokens con expiración de **8 horas** (`expiresIn: '8h'`)
- Verificación de firma criptográfica en cada request
- Payload del token incluye `clinica_id` para aislamiento multi-tenant
- Respuestas de error genéricas (no revelan si el token está expirado vs. inválido)

Payload del JWT:

```
// Token de Clínica (Administrador)
{ id, tipo: 'clinica', clinica_id, nombre }

// Token de Empleado
{ id, tipo: 'empleado', clinica_id, rol_id, nombre }
```

2.2 Hashing de Contraseñas con bcrypt

Archivos: `src/routes/clinicas.js`, `src/routes/empleados.js`

```
// Registro – hash antes de guardar
const password_hash = await bcrypt.hash(password, SALT_ROUNDS); // 10 rounds

// Login – comparación segura (timing-safe)
const passwordOk = await bcrypt.compare(password, clinica.password_hash);
if (!passwordOk) {
  return res.status(401).json({ error: 'Credenciales incorrectas' });
}
```

Buenas prácticas aplicadas:

- Nunca se almacena la contraseña en texto plano
- `bcrypt.compare()` es resistente a ataques de timing
- 10 rounds de salt (estándar de la industria para 2026)
- Contraseñas temporales generadas con entropía suficiente

Generador de contraseñas temporales:

```
function generarPasswordTemporal() {
  const chars = 'ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz23456789';
  const especiales = '!@#$%*';
  let pwd = '';
  for (let i = 0; i < 8; i++) {
    pwd += chars[Math.floor(Math.random() * chars.length)];
  }
  pwd += especiales[Math.floor(Math.random() * especiales.length)];
  return pwd; // 9 caracteres: 8 alfanuméricos + 1 especial
}
```

2.3 Control de Acceso Basado en Roles (RBAC)

Archivo: `src/middleware/authMiddleware.js`

El sistema implementa 4 niveles de autorización:

Middleware	Acceso Permitido	Uso
<code>authMiddleware</code>	Cualquier usuario autenticado	Todas las rutas protegidas
<code>soloClinica</code>	Solo tipo 'clinica' (Admin)	Gestión de empleados, roles, perfil
<code>soloEmpleado</code>	Solo tipo 'empleado'	Dashboard de empleado
<code>clinicaOVeterinario</code>	Admin O Veterinario (rol_id=2)	Operaciones clínicas sensibles

```
// Ejemplo de uso en rutas
router.get('/', authMiddleware, soloClinica, handler);           // Solo admin
router.delete('/:id', clinicaOVeterinario, handler);           // Admin o Vet
router.use(authMiddleware); router.get('/', handler);           // Cualquier auth
```

Verificación de pertenencia al tenant (anti-IDOR):

```
// Antes de cualquier operación, se verifica que el recurso
// pertenece a la clínica del usuario autenticado
db.query(
  'SELECT id FROM expediente WHERE id = ? AND clinica_id = ?',
  [expediente_id, req.user.clinica_id], // Double-check
  (errCheck, rows) => {
    if (rows.length === 0) return res.status(404).json({
      error: 'Expediente no encontrado o no pertenece a esta clínica'
    });
    // Proceder con la operación
  }
);
```

2.4 Aislamiento Multi-Tenant

Patrón: Row-Level Security implementado en la capa de aplicación.

Cada query que accede a datos sensibles incluye el filtro `clinica_id`:

```
// Pacientes – siempre filtrado por clínica
db.query(BASE_SQL + ' WHERE p.clinica_id = ?', [req.user.clinica_id], ...);

// Empleados – siempre filtrado por clínica
db.query('SELECT ... FROM empleados WHERE clinica_id = ?', [req.user.clinica_id], ...);

// Inventario – siempre filtrado por clínica
```

```
db.query('SELECT * FROM inventario WHERE clinica_id = ?', [req.user.clinica_id], ...);
```

Verificación de roles por clínica:

```
// Al asignar un rol a un empleado, se verifica que el rol
// pertenece a la misma clínica (evita escalada de privilegios)
db.query(
  'SELECT id FROM roles WHERE id = ? AND clinica_id = ?',
  [rol_id, req.user.clinica_id],
  (err, rolRows) => {
    if (rolRows.length === 0) {
      return res.status(400).json({
        error: 'El rol_id no existe o no pertenece a tu clínica'
      });
    }
  }
);
```

2.5 Validación de Cuentas Suspendidas

```
// clinicas.js – Verificación de estado de la clínica
if (!clinica.activa) {
  return res.status(403).json({
    error: 'Esta clínica está suspendida. Contacte al administrador del sistema.'
  });
}

// empleados.js – Verificación de estado del empleado Y de su clínica
if (!empleado.activo) {
  return res.status(403).json({
    error: 'Tu cuenta está desactivada. Contacta al administrador de tu clínica.'
  });
}

if (!empleado.clinica_activa) {
  return res.status(403).json({
    error: 'La clínica está suspendida. Contacta al administrador del sistema.'
  });
}
```

2.6 Protección de Rutas en el Frontend

Archivo: src/App.js

```
// ProtectedRoute – Redirige al Login si no hay sesión activa
function ProtectedRoute({ children }) {
  const { isLoggedIn } = useAuth();
  const { pathname } = useLocation();

  if (!isLoggedIn) {
    return <Navigate to="/login" state={{ from: pathname }} replace />;
  }
  return children;
}

// PublicRoute – Redirige al Dashboard si ya hay sesión activa
// (evita que un usuario logueado vea el Login)
function PublicRoute({ children }) {
  const { isLoggedIn } = useAuth();
  if (isLoggedIn) {
    return <Navigate to="/" replace />;
  }
  return children;
}
```

2.7 Gestión Segura de Sesión en el Frontend

Archivo: `src/context/AuthContext.js`

```
// Persistencia segura en localStorage
function persistSession(session) {
  if (session) {
    localStorage.setItem(SESSION_KEY, JSON.stringify(session));
    // Inyección automática del token en todos los requests
    API.defaults.headers.common['Authorization'] = `Bearer ${session.token}`;
  } else {
    localStorage.removeItem(SESSION_KEY);
    // Limpieza del header al cerrar sesión
    delete API.defaults.headers.common['Authorization'];
  }
}

// Logout completo – limpia estado, localStorage y headers
const logout = useCallback(() => {
  setSession(null);
  localStorage.removeItem(SESSION_KEY);
  delete API.defaults.headers.common['Authorization'];
  window.location.href = '/login'; // Redirección forzada
}, []);
```

2.8 Configuración de Variables de Entorno

```
// Backend – Nunca hardcoded en producción
const db = mysql.createPool({
  host:      process.env.DB_HOST,
  user:      process.env.DB_USER,
  password:  process.env.DB_PASS,
  database:  process.env.DB_NAME,
});

const JWT_SECRET = process.env.JWT_SECRET || 'saas_vet_secret_2026';
const PORT = process.env.PORT || 3000;

// Frontend – URL configurable por entorno
const API = axios.create({
  baseURL: process.env.REACT_APP_API_URL || 'http://localhost:4000/api',
});
```

3. Manejo de Errores

3.1 Middleware Global de Errores

Archivo: `src/middleware/errorHandler.js`

```
function errorHandler(err, req, res, next) {
  // Evitar enviar respuesta si ya se inició el streaming
  if (res.headersSent) return next(err);

  const status = err.status || err.statusCode || 500;
  const message = err.message || 'Error interno del servidor';

  // Log diferenciado por entorno
  if (process.env.NODE_ENV !== 'production') {
    console.error(`[ERROR ${status}] ${req.method} ${req.originalUrl} →`, err);
  } else {
    // En producción: no exponer stack trace
    console.error(`[ERROR ${status}] ${req.method} ${req.originalUrl} →
    ${message}`);
  }

  res.status(status).json({
    error: message,
    // Stack trace solo en desarrollo
    ...(process.env.NODE_ENV !== 'production' && err.stack ? { stack: err.stack }
    : {}),
  });
}
```

Características:

- Respuesta JSON uniforme para todos los errores
- Stack trace solo en desarrollo (no expuesto en producción)
- Log detallado en consola con método HTTP y URL
- Manejo de `headersSent` para evitar errores de "headers already sent"

3.2 Manejo de Errores por Capa

Backend — Errores de Base de Datos

```
// Patrón consistente en todas las rutas
db.query(sql, params, (err, rows) => {
  if (err) return res.status(500).json({ error: err.message });
  // Lógica de negocio
});

// Manejo de errores de FK (integridad referencial)
router.delete('/:id', authMiddleware, soloClinica, (req, res) => {
  db.query('DELETE FROM roles WHERE id = ? AND clinica_id = ?', [...], (err,
result) => {
    if (err) {
      // Error específico de FK – mensaje descriptivo para el usuario
      if (err.code === 'ER_ROW_IS_REFERENCED_2') {
        return res.status(409).json({
          error: 'No se puede eliminar el rol porque hay empleados asignados a
él.',
        });
      }
      return res.status(500).json({ error: err.message });
    }
    if (result.affectedRows === 0) return res.status(404).json({ error: 'Rol no
encontrado' });
    res.json({ mensaje: 'Rol eliminado correctamente' });
  });
});
```

Backend — Errores en Generación de PDFs

```
// reportsController.js – Manejo defensivo de logo
const addHeader = (doc, title) => {
  try {
    doc.image(LOGO_PATH, 50, 45, { width: 60 });
  } catch (e) {
    // Si el logo no carga, continuar sin él (degradación elegante)
  }
  // El resto del header se genera normalmente
};
```

Frontend — Manejo de Errores en Componentes

```
// Dashboard.js – Estados de carga y error
const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  API.get('/dashboard/clinica')
    .then(r => setData(r?.data ?? null))
    .catch(e => setError(e?.response?.data?.error || 'Error al cargar el dashboard'))
    .finally(() => setLoading(false));
}, []);

// Renderizado condicional por estado
if (loading) return <SkeletonLoader />;
if (error)   return <ErrorBanner message={error} />;
return <DashboardContent data={data} />;
```

Frontend — Manejo de Errores en Login

```
// Login.js – Extracción segura del mensaje de error
try {
  await loginClinica(email, password);
  navigate('/', { replace: true });
} catch (err) {
  const msg =
    err?.response?.data?.error ||
    'Error al iniciar sesión. Verifica tus credenciales.';
  setError(msg);
} finally {
  setLoading(false);
}
```

3.3 Códigos HTTP Semánticos

El sistema usa códigos HTTP apropiados para cada situación:

Código	Uso en ANA-VET
200 OK	Operación exitosa (GET, PUT)
201 Created	Recurso creado exitosamente (POST)
400 Bad Request	Campos requeridos faltantes
401 Unauthorized	Token no proporcionado, inválido o expirado

Código	Uso en ANA-VET
403 Forbidden	Acceso denegado por rol o cuenta suspendida
404 Not Found	Recurso no encontrado
409 Conflict	Email duplicado, FK violation
500 Internal Server Error	Error de base de datos u otro error interno

3.4 Validación de Datos de Entrada

```
// Validación en pacientes.js
if (!tutor_id || !nombre || !especie || !sexo) {
  return res.status(400).json({
    error: 'Faltan campos requeridos: tutor_id, nombre, especie, sexo'
  });
}

// Validación en recibos.js – validación de items
for (const item of items) {
  if (!item.nombre_servicio || item.precio_unitario === undefined ||
  !item.cantidad) {
    return res.status(400).json({
      error: 'Cada item requiere: nombre_servicio, precio_unitario, cantidad',
    });
  }
}

// Validación de status en recibos
if (status && !['borrador', 'finalizado'].includes(status)) {
  return res.status(400).json({ error: 'status debe ser: borrador o finalizado'
});
}
```

4. Refactorización Realizada

4.1 Refactorización: Callbacks a Async/Await con Promise Wrapper

Antes (patrón callback):

```
// Múltiples queries anidadas – "callback hell"
db.query(sql1, params1, (err1, rows1) => {
  if (err1) return res.status(500).json({ error: err1.message });
  db.query(sql2, params2, (err2, rows2) => {
    if (err2) return res.status(500).json({ error: err2.message });
    db.query(sql3, params3, (err3, rows3) => {
      // ...
    });
  });
});
```

```
});  
});
```

Después (async/await con Promise.all):

```
// connection.js – wrapper Promise  
function query(sql, params) {  
  return new Promise((resolve, reject) => {  
    db.query(sql, params, (err, rows) => {  
      if (err) return reject(err);  
      resolve(rows);  
    });  
  });  
}  
  
// dashboard.js – queries paralelas con Promise.all  
const [rowsIngresos, rowsPacientes, rowsConsultasHoy, rowsScorecard] = await  
Promise.all([  
  query(sqlIngresos, [clinica_id]),  
  query(sqlPacientes, [clinica_id]),  
  query(sqlConsultasHoy, [clinica_id]),  
  query(sqlScorecard, [clinica_id, clinica_id, clinica_id, clinica_id]),  
]);
```

Beneficio: Eliminación del callback hell, código más legible, ejecución paralela de queries (mejor rendimiento).

4.2 Refactorización: Corrección de Bug en Reporte de Consultas

Antes (bug):

```
// Bug: se intentaba acceder a dx_definitivo en la tabla consulta  
// pero ese campo NO existe en consulta, está en expediente  
const query = `SELECT c.id, c.fecha, c.dx_definitivo FROM consulta c ...`;
```

Después (corregido):

```
// Bug fix documentado en el código  
// Bug fix: la tabla consulta NO tiene dx_definitivo; ese campo está en  
// expediente.  
const query = `  
  SELECT c.id, c.fecha, c.motivo, c.dx_presuntivo,  
         e.dx_definitivo, -- Correcto: viene de expediente  
         p.nombre as paciente_nombre, t.nombre as tutor_nombre  
  FROM consulta c  
  LEFT JOIN expediente e ON c.expediente_id = e.id`;
```

```
...  
`;  
`;
```

4.3 Refactorización: Normalización de Fechas en Recibos

Antes:

```
// El frontend enviaba fechas en formato DD/MM/YYYY  
// MySQL requiere YYYY-MM-DD – causaba errores silenciosos  
db.query('INSERT INTO recibo ... VALUES (?, ...)', [fecha, ...]);
```

Después:

```
// Normalización explícita con documentación  
let fechaNormalizada = fecha;  
if (fecha && /^{\d{2}}\/{\d{2}}\/{\d{4}}$/.test(fecha)) {  
  const [d, m, y] = fecha.split('/');  
  fechaNormalizada = `${y}-${m}-${d}`; // DD/MM/YYYY → YYYY-MM-DD  
}
```

4.4 Refactorización: Corrección de Nombre de Tabla en Inventario

Antes (bug):

```
// Error: la tabla se llama 'empleados' (plural), no 'empleado'  
const sql = `SELECT sr.*, e.nombre FROM solicitud_reabastecimiento sr  
  LEFT JOIN empleado e ON sr.empleado_id = e.id`;  
// Resultado: error SQL "Table 'empleado' doesn't exist"
```

Después (corregido con comentario):

```
// FIX: la tabla de empleados se llama 'empleados' (plural), no 'empleado'  
const sql = `SELECT sr.*, CONCAT(e.nombre, ' ', e.apellidos) AS solicitado_por  
  FROM solicitud_reabastecimiento sr  
  LEFT JOIN empleados e ON sr.empleado_id = e.id  
  WHERE sr.clinica_id = ?`;
```

4.5 Refactorización: Generación de Correos Corporativos Únicos

Antes:

```
// Problema: si dos empleados tenían el mismo nombre, el email colisionaba
const email = `${nombre}.${apellido}@anavet-${clinica_id}.com`;
// Resultado: error de UNIQUE constraint en la BD
```

Después:

```
// Solución: algoritmo de resolución de conflictos
function limpiarTexto(str) {
  return str
    .normalize('NFD')
    .replace(/[\u0300-\u036f]/g, '') // Elimina acentos
    .replace(/^[a-zA-Z0-9]/g, '') // Solo alfanumérico
    .toLowerCase();
}

function resolverCorreoUnico(baseCorreo, clinica_id, callback) {
  // Busca todos los correos similares en la BD
  db.query('SELECT email FROM empleados WHERE email LIKE ?',
    [`${localBase}%${dominio}`], (err, rows) => {
    if (rows.length === 0) return callback(null, baseCorreo);
    // Agrega sufijo numérico incremental hasta encontrar uno libre
    let contador = 1;
    let candidato = `${localBase}${contador}${dominio}`;
    while (existentes.includes(candidato)) { contador++; }
    return callback(null, candidato);
  });
}
```

4.6 Refactorización: Actualización de Stock al Completar Solicitud

Antes:

```
// Solo se actualizaba el status de la solicitud
// El stock del inventario no se actualizaba automáticamente
db.query('UPDATE solicitud_reabastecimiento SET status = ? WHERE id = ?', [status, id]);
```

Después:

```
// Al marcar como 'completado', se actualiza el stock automáticamente
if (status === 'completado') {
  const cantidad = solicitud.cantidad ?? 1;
  const updateStockSql = `
    UPDATE inventario
    SET stock = stock + ?
```

```

    WHERE LOWER(nombre) = LOWER(?) AND clinica_id = ?
  `;
  db.query(updateStockSql, [cantidad, solicitud.producto_nombre, clinica_id],
(err3) => {
    if (err3) {
      // Loguear el error pero no revertir el status
      console.error('Error actualizando stock en inventario:', err3.message);
      return res.json({
        message: 'Solicitud marcada como completada, pero hubo un error al
actualizar el stock',
        stockError: err3.message
      });
    }
    res.json({ message: 'Solicitud completada y stock actualizado en inventario'
});
  });
}

```

4.7 Refactorización: Helpers de Formato Seguro en Dashboard

Antes:

```

// Acceso directo a propiedades – podía causar errores si la API devolvía null
<p>{data.kpis.ingresos_mes}</p>
<p>{data.scorecard_empleados[0].nombre}</p>

```

Después:

```

// Helpers defensivos que nunca lanzan excepciones
function safeCurrency(val) {
  const n = Number(val ?? 0);
  return isNaN(n) ? '$0.00' : `$$${n.toLocaleString('es-MX', {...})}`;
}

function safeInt(val) {
  const n = Number(val ?? 0);
  return isNaN(n) ? 0 : n;
}

function safeStr(val, fallback = '-') {
  return (val !== null && val !== undefined && String(val).trim() !== '')
    ? String(val) : fallback;
}

function safeDate(val) {
  try {
    const d = new Date(val);
    if (isNaN(d.getTime())) return '-';
  }
}

```

```

    return d.toLocaleDateString('es-MX', {...});
  } catch { return '-'; }
}

// Uso seguro
<p>{safeCurrency(kpis?.ingresos_mes)}</p>
<p>{safeStr(emp?.empleado_nombre, 'Sin nombre')}</p>

```

5. Buenas Prácticas de Código

5.1 Comentarios y Documentación Inline

```

// authMiddleware.js - JSDoc completo
/**
 * Middleware de autenticación JWT.
 * Verifica el token en el header Authorization: Bearer <token>
 * Inyecta req.user con { id, tipo, clinica_id }
 *   - tipo: 'clinica' | 'empleado'
 */
function authMiddleware(req, res, next) { /* ... */ }

// connection.js - JSDoc con tipos
/**
 * Promise wrapper for db.query - allows async/await usage.
 * @param {string} sql
 * @param {Array} params
 * @returns {Promise<Array>}
 */
function query(sql, params) { /* ... */ }

// Comentarios de sección en rutas
// =====
// POST /api/clinicas/registro
// Registra una nueva clínica en el sistema SaaS
// Body: { nombre, email, password, telefono?, direccion? }
// =====

```

5.2 Nomenclatura Consistente

Convención	Aplicación
camelCase	Variables, funciones, parámetros JS
snake_case	Columnas de BD, campos de API JSON
PascalCase	Componentes React, clases
UPPER_SNAKE_CASE	Constantes (JWT_SECRET, SALT_ROUNDS, SESSION_KEY)

Convención	Aplicación
kebab-case	Nombres de archivos de rutas (<code>servicios-catalogo.js</code>)

5.3 Uso de COALESCE para Actualizaciones Parciales

```
// Patrón COALESCE – permite actualizar solo los campos enviados
// sin necesidad de enviar todos los campos en el body
db.query(
  `UPDATE clinicas SET
    nombre    = COALESCE(?, nombre),
    telefono  = COALESCE(?, telefono),
    direccion = COALESCE(?, direccion),
    logo_url  = COALESCE(?, logo_url)
  WHERE id = ?`,
  [nombre || null, telefono || null, direccion || null, logo_url || null, id]
);
```

Beneficio: Permite actualizaciones parciales (PATCH-like) sin sobrescribir campos no enviados.

5.4 Soft Delete vs. Hard Delete

```
// tutores.js – Soft Delete (preserva historial)
router.delete('/:id', clinicaOVeterinario, (req, res) => {
  db.query(
    "UPDATE tutor SET estatus = 'inactivo' WHERE id = ? AND clinica_id = ?",
    [tutorId, clinicaId],
    // ...
  );
});

// Vetar tutor (estado especial)
router.put('/:id/vetar', clinicaOVeterinario, (req, res) => {
  db.query(
    "UPDATE tutor SET estatus = 'vetado', vetado = 1 WHERE id = ? AND clinica_id = ?",
    // ...
  );
});

// recibos.js – Hard Delete solo permitido en estado 'borrador'
router.delete('/:id', (req, res) => {
  if (recibo.status !== 'borrador') {
    return res.status(409).json({
      error: 'Solo se pueden eliminar recibos en estado borrador',
    });
  }
});
```

```
db.query('DELETE FROM recibo WHERE id = ? AND clinica_id = ?', [...]);
});
```

5.5 Uso de `INSERT IGNORE` para Tablas Puente

```
// Evita errores de duplicados en tablas N:M
db.query(
  'INSERT IGNORE INTO cirugia_empleados (cirugia_id, empleado_id) VALUES (?, ?)',
  [cirugiaId, empId],
  // ...
);

db.query(
  'INSERT IGNORE INTO hospitalizacion_empleados (hospitalizacion_id, empleado_id)
VALUES (?, ?)',
  [hospId, empId],
  // ...
);
```

5.6 Memoización con `useCallback` y `useMemo`

```
// AuthContext.js – Funciones memoizadas para evitar re-renders innecesarios
const loginClinica = useCallback(async (email, password) => { /* ... */ }, []);
const loginEmpleado = useCallback(async (email, password) => { /* ... */ }, []);
const logout = useCallback(() => { /* ... */ }, []);

// SelectedAnimalContext.js – Valores computados memoizados
const selectedAnimalColor = useMemo(
  () => getAnimalColor(selectedAnimal),
  [selectedAnimal] // Solo recalcula cuando cambia el animal
);

const selectedAnimalTextColor = useMemo(
  () => (isDarkColor(selectedAnimalColor) ? '#FFFFFF' : '#1F2937'),
  [selectedAnimalColor]
);
```

5.7 Cleanup de Efectos (Prevención de Memory Leaks)

```
// NotificacionesBell.js – Limpieza de interval
useEffect(() => {
  fetchNotificaciones();
  const interval = setInterval(fetchNotificaciones, 5 * 60 * 1000);
  return () => clearInterval(interval); // Cleanup al desmontar
});
```



```
}, [fetchNotificaciones]);

// Limpieza de event listener
useEffect(() => {
  function handleClickOutside(e) { /* ... */ }
  if (open) {
    document.addEventListener('mousedown', handleClickOutside);
  }
  return () => document.removeEventListener('mousedown', handleClickOutside);
}, [open]);
```

5.8 Validación de Arrays antes de Operaciones

```
// Patrón defensivo para arrays de la API
const scorecard = Array.isArray(data?.scorecard_empleados) ?
data.scorecard_empleados : [];
const ingresosRecientes = Array.isArray(data?.ingresos_recientes) ?
data.ingresos_recientes : [];
const alertasInventario = Array.isArray(data?.alertas_inventario) ?
data.alertas_inventario : [];

// En notificaciones
setNotificaciones(Array.isArray(data) ? data : []);
```

5.9 Pool de Conexiones con Límite

```
// connection.js – Configuración del pool
const db = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  waitForConnections: true,    // Espera si no hay conexiones disponibles
  connectionLimit: 10,        // Máximo 10 conexiones simultáneas
  queueLimit: 0,               // Sin límite en la cola de espera
});
```

5.10 Generación de Códigos Únicos para Tutores

```
// tutores.js – Código único con timestamp + random
const codigo = `TUT-${Date.now()}-${Math.floor(Math.random() * 9000 + 1000)}`;
// Ejemplo: TUT-1779239761255-7902
// Garantiza unicidad práctica sin consulta adicional a la BD
```

6. Métricas de Calidad

6.1 Cobertura de Funcionalidades

Módulo	Endpoints	Autenticación	Validación	Multi-tenant
Clínicas	5	✓	✓	✓
Empleados	6	✓	✓	✓
Roles	5	✓	✓	✓
Tutores	5	✓	✓	✓
Pacientes	5	✓	✓	✓
Expedientes	2	✓	✓	✓
Consultas	3	✓	✓	✓
Hospitalizaciones	3	✓	✓	✓
Cirugías	3	✓	✓	✓
Vacunas	3	✓	✓	✓
Recibos	5	✓	✓	✓
Inventario	5	✓	✓	✓
Dashboard	2	✓	N/A	✓
Notificaciones	1	✓	N/A	✓
Calculadora	2	✗ (público)	N/A	N/A
Reportes PDF	7	✓	N/A	✓

Total endpoints: ~62 endpoints REST

6.2 Complejidad Ciclomática Estimada

Archivo	Funciones	Complejidad Promedio	Nivel
authMiddleware.js	4	Baja (2-3)	✓ Bajo
errorHandler.js	1	Baja (2)	✓ Bajo

Archivo	Funciones	Complejidad Promedio	Nivel
connection.js	2	Baja (2)	✓ Bajo
clinicas.js	5	Media (4-6)	✓ Aceptable
empleados.js	6	Alta (7-10)	⚠ Moderado
dashboard.js	2	Media (4-5)	✓ Aceptable
recibos.js	5	Media (5-7)	✓ Aceptable
inventario.js	5	Media (4-6)	✓ Aceptable
reportsController.js	8	Baja (3-4)	✓ Bajo
AuthContext.js	4	Media (4-5)	✓ Aceptable
Dashboard.js	12	Media (4-6)	✓ Aceptable

6.3 Dependencias del Backend

Dependencia	Versión	Propósito	Seguridad
express	^5.2.1	Framework HTTP	✓ Última versión
mysql2	^3.22.3	Driver MySQL con prepared statements	✓ Previene SQL injection
bcryptjs	^3.0.3	Hashing de contraseñas	✓ Estándar industria
jsonwebtoken	^9.0.3	Autenticación JWT	✓ Estándar industria
dotenv	^17.4.2	Variables de entorno	✓ Buena práctica
cors	^2.8.6	Control de CORS	✓ Configurado
pdfkit	^0.18.0	Generación de PDFs	✓ Sin vulnerabilidades conocidas
nodemon	^3.1.14	Hot reload (dev only)	✓ Solo desarrollo

6.4 Prevención de SQL Injection

El sistema usa **prepared statements** (queries parametrizadas) en el 100% de las operaciones de base de datos:

```
// ✓ CORRECTO – Parámetros separados del SQL
db.query('SELECT * FROM clinicas WHERE email = ?', [email], callback);
db.query('INSERT INTO paciente (...) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)', [...params],
callback);

// ✗ NUNCA se hace esto en el código
// db.query(`SELECT * FROM clinicas WHERE email = '${email}'`); // Vulnerable
```

Beneficio: mysql2 con prepared statements previene completamente la inyección SQL.

6.5 Métricas de Líneas de Código (LOC)

Componente	Archivos	LOC Estimado
Backend — Rutas	16 archivos	~1,800 LOC
Backend — Middleware	2 archivos	~92 LOC
Backend — Controladores	1 archivo	~553 LOC
Backend — DB/Config	2 archivos	~53 LOC
Frontend — Páginas	18 archivos	~3,500 LOC
Frontend — Componentes	5 archivos	~800 LOC
Frontend — Calculadoras	10 archivos	~2,000 LOC
Frontend — Contextos	4 archivos	~400 LOC
Frontend — Utils	1 archivo	~102 LOC
Total estimado	~59 archivos	~9,300 LOC

6.6 Análisis de Deuda Técnica

Área	Deuda Identificada	Impacto	Prioridad
Autenticación	JWT_SECRET con fallback hardcoded	Medio	Alta
Validación	Sin librería de validación (Joi/Zod)	Bajo	Media
Testing	Sin tests unitarios ni de integración	Alto	Alta
Rate Limiting	Sin límite de requests por IP	Medio	Media
HTTPS	No configurado en el código (depende del deploy)	Alto	Alta
Logging	Solo <code>console.error</code> , sin logger estructurado	Bajo	Baja
Paginación	Algunos endpoints sin paginación	Bajo	Baja

7. Degradación Elegante (Graceful Degradation)

El sistema implementa degradación elegante en varios puntos:

```
// 1. Logo en PDFs – continúa sin logo si no se puede cargar
try {
  doc.image(LOGO_PATH, 50, 45, { width: 60 });
} catch (e) {
  // Continúa sin logo
}
```

```
// 2. Roles por defecto – advertencia sin fallo si no se pueden crear
db.query('INSERT INTO roles ...', [rolesDefault], (err3) => {
  if (err3) console.warn('Advertencia: no se pudieron crear roles por defecto:',
err3.message);
  // La clínica se crea de todas formas
});

// 3. Stock en inventario – solicitud completada aunque falle la actualización de
stock
if (err3) {
  console.error('Error actualizando stock en inventario:', err3.message);
  return res.json({
    message: 'Solicitud marcada como completada, pero hubo un error al actualizar
el stock',
    stockError: err3.message
  });
}

// 4. Notificaciones – array vacío si falla la carga
setNotificaciones(Array.isArray(data) ? data : []);

// 5. Sesión – null si localStorage está corrupto
function loadSession() {
  try {
    const raw = localStorage.getItem(SESSION_KEY);
    return raw ? JSON.parse(raw) : null;
  } catch {
    return null; // Sesión corrupta → null (fuerza re-login)
  }
}
```

8. Resumen de Calidad

Categoría	Implementación	Nivel
Autenticación JWT	✅ Completa con expiración	Alto
Hashing de contraseñas	✅ bcrypt 10 rounds	Alto
Control de acceso RBAC	✅ 4 niveles de autorización	Alto
Aislamiento multi-tenant	✅ En todas las queries	Alto
Validación de entrada	✅ Campos requeridos verificados	Medio
Manejo de errores	✅ Middleware global + por ruta	Alto
Prevención SQL injection	✅ Prepared statements 100%	Alto
Protección de rutas frontend	✅ ProtectedRoute + PublicRoute	Alto
Variables de entorno	✅ dotenv configurado	Alto

Categoría	Implementación	Nivel
Degradación elegante	✓ Múltiples puntos	Medio
Documentación de código	✓ JSDoc + comentarios inline	Medio
Soft delete	✓ Tutores con estatus	Medio
Tests automatizados	✗ No implementados	Bajo
Rate limiting	✗ No implementado	Bajo

Documentación generada a partir del código fuente del proyecto ANA-VET · Mayo 2026